

The semantics and syntaxes for Rust

Rust supports all types of semantics with a very easy and flexible syntax. It provides many different types of syntaxes for basic coding, variable binding, conditional statements, vectors and much more.

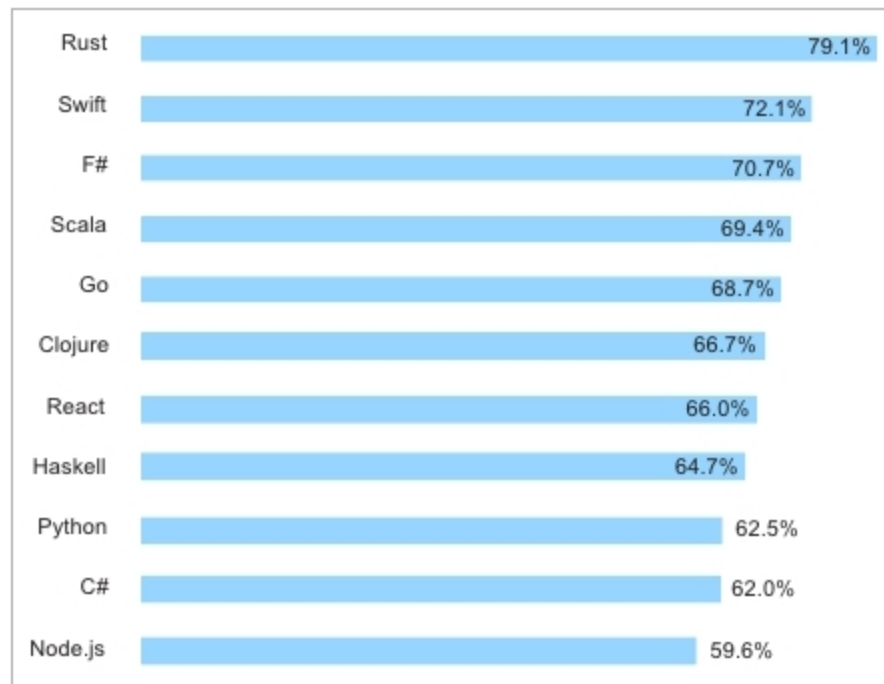


Figure 1: Popularity of Rust among developers (Source: <https://insights.stackoverflow.com>)

Rust offers systems of ownership and easy memory management, making it quite safe from memory loss. As it supports the ownership features of semantics, it can also reference, borrow and pass on the ownership and rights of access of any data member or function in the programming language. With the feature of type casting, we can also make a type alias in the code with just the simple keyword 'type'.

Apart from this, all the basic features are perfectly stable. These include access authorisation, raw pointers, crates and modules, constants, universal function-calling, patterns, generics, mutability and many others.

Writing effective Rust code

Rust is a basic, low-level programming language and hence can bind easily with the systems of any type of architecture. This is why it is known as a systems level programming language. Rust has the semantics, easy syntax and coding format that help to provide iterators and concurrency formats with a simple syntax, and hence it increases the efficiency of the program as well as its runtime accuracy and precision. It has a built-

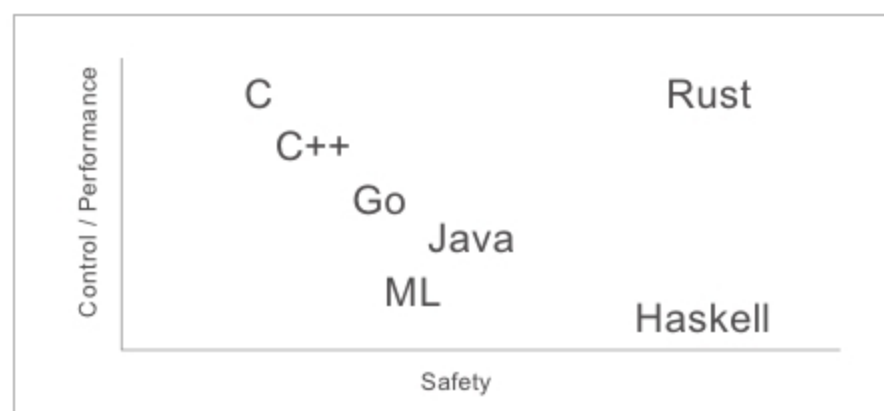


Figure 2: Graphical comparison of Rust

The pros and cons of Rust

Advantages of Rust	Disadvantages of Rust
It is memory safe and secure. This means that it has good backup support, the memory occupation is limited, and the memory access becomes easier and safer to rely upon.	It does not support patching like other languages.
It can be compiled and run quickly.	It is not Web scalable.
It is very much concurrent. This means that it has a consistent performance wherever it is used... be it any type of logic or background. Its performance, execution and efficiency are not affected due to the environment in which it is going to be used.	Being new, its syntax is not very familiar.
It has a built-in management system.	It has no standard libraries like other languages, which is both a boon and a bane.
It is an independent language. Though it is compatible with many other languages, platforms and environments, it still holds its own special position in the code, standing out differently and not depending on other languages.	Its garbage value randomisation is not recommendable.

in error-handling feature, which makes it easy for even a lay person to easily use this language. Rust has no standard form of library; therefore, there are no hard and fast rules that need to be followed in order to write efficient program code.

The best part about Rust is that it does not require users to know any other type of coding language, and can be learned and used effectively within a short period of time. Rust is an object-oriented programming (OOP) language and is data oriented. It supports all the features of OOP such as polymorphism, inheritance, encapsulation and abstraction. **END** 🐧

References

- [1] <https://doc.rust-lang.org/book/first-edition/getting-started.html>
- [2] <https://medium.com/learning-rust/rust-basics-e73304ab35c7>
- [3] <https://thenewstack.io/safer-future-rust/>

By: Dhruvil Trivedi

The author is FOSS enthusiast and can be contacted at dhruviltrivedi2610@gmail.com.